

# Rank 2 Vulnerability Fix — AI Session Log

## AES-128 Peripheral: FSM and Round-Counter Integrity Protection

Waylon Woo with Claude Opus 4.6 (1M context)

2026-04-12

### Session Log: Rank 2 Vulnerability Fix

**Environment:** Claude Code v2.1.104 (Opus 4.6, 1M context, high-effort mode) **Working directory:** ~/repo/projects/AI-HDL/main **Outcome:** Fix merged as commit 491d2dc on branch DP3.

---

#### Turn 1 — Read the security report, focus on Rank 2

**User:** In DP3\_submission read security eval and tell me about Rank 2 vulnerability.

*(Claude extracted text from the 13-page security\_report.pdf and located the Rank 2 section.)*

**Claude:**

#### Rank 2 — CRITICAL: FSM and Round Counter Have No Integrity Protection

**Problem.** The AES state machine is encoded in just 2 bits, and the round counter is a plain 5-bit number. Neither has parity, redundancy, or illegal-state detection.

##### Single-bit-flip consequences:

- COUNTING (01) → IDLE (00): encryption aborts silently; the output register leaks a partial result that can reveal key material.
- COUNTING (01) → DONE (10): encryption “finishes” early and emits an incomplete ciphertext — classic DFA setup where an attacker recovers the key by comparing the faulty output to a correct ciphertext.
- Counter 19 → 21: skips an AES round entirely, producing weak 9-round output.

**Attack cost.** A ~\$200 ChipWhisperer can inject voltage glitches with nanosecond precision. Because timing is fixed at 22 cycles, the attacker knows exactly when round 9 executes. Two to four successful glitches recover the full 128-bit key.

**Report’s recommended fix:** one-hot FSM with illegal-state detector; shadow counter alongside the round counter with mismatch detection; on fault, zero all registers and refuse to output anything.

**Effort estimate:** Medium — ~24 extra flip-flops plus comparison logic.

---

#### Turn 2 — What specific RTL changes would fix it?

**User:** What changes would you make to fix this vulnerability?

(Claude read AES128/Updated\_Control\_Unit.v to ground the plan in the actual code.)

**Claude:** Looking at the file, the real code is even slightly worse than the report implies: line 94–97 has `else begin state <= DONE; end` as a silent fallback for out-of-range counts — which is the exact attacker-favoring transition. Concrete changes:

1. **One-hot FSM with a sticky FAULT state.** Replace the 2-bit binary encoding (lines 20–25) with 4-bit one-hot {IDLE, RUN, DONE, FAULT}. Every cycle, check that exactly one bit is set — any single flip produces a 0-hot or 2-hot value that's detected and latches FAULT.
2. **Kill the silent-DONE fallback.** `else begin state <= DONE; end` is a gift to a fault attacker — it satisfies `done=1` from an illegal count. Change to `state <= FAULT;`.
3. **Shadow down-counter for count.** Add a second counter initialized such that `count + count_shadow == constant` invariantly. Any single bit-flip in either register fails the invariant.
4. **Protect the column-phase register.** Duplicate phase (`phase_a`, `phase_b`) and compare each cycle; `mismatch <= FAULT`.
5. **Terminal-state sanity check.** Before asserting `done`, require `count == final && phase == 3 && shadow == 1`. If not met, go FAULT instead of DONE.
6. **FAULT behavior that actually protects the key.** FAULT must force `done=0`, zero the ciphertext output register in the encryption engine, and optionally zero the round-key register. A controller that just halts while the datapath still holds a partial ciphertext is useless.
7. **Tighten the reset path.** Currently `reset = reset_n || reset_s` and `posedge reset` — either reset yanks everything. If `reset_s` is attacker-reachable (register-mapped), it can also wipe the FAULT latch. Consider making FAULT clearable only by the async `reset_n`.

**Cost:** ~8–12 flip-flops for shadow + phase duplicate, 2 extra bits for one-hot state, a comparator, and the FAULT fan-out.

---

## Turn 3 — Justify the phase-duplicate defense

**User:** Why is 4 needed?

**Claude:** Because phase is part of the same round-tracking logic — if you harden count but leave phase bare, the attacker just retargets the glitch.

Look at the transition logic: `count <= count + 1` fires when `phase == 2'd3`. A single bit-flip that forces phase from 00 to 11 one cycle into a round causes the controller to “complete” that round after 1 cycle instead of 4 — meaning 3 of the 4 SubBytes columns never get processed. The round advances, the shadow counter still agrees with the main counter (both incremented legitimately), and the FAULT detector never fires.

Net effect: same class of attack as a round-skip, same DFA math, same ~2–4 glitches to key recovery — just aimed at a different 2-bit register. Hardening only the counter moves the attack from “glitch the counter” to “glitch phase,” which is strictly easier (smaller register, same timing knowledge).

Short version: count and phase jointly define “where are we in the encryption.” Both have to be integrity-protected or neither is.

---

## Turn 4 — Implement the fix

**User:** Great. Implement the changes needed to fix Rank 2.

*(Claude read Updated\_Encryption\_Engine\_4\_S-boxes.v and Updated\_Peripheral.v to plan the integration.)*

**Claude:** I can do this cleaner than planned. Instead of adding a fault port to the engine, I'll have the control unit expose fault, and the peripheral will OR it into the engines' reset input. The engines already zero their data registers on async reset — so no engine code changes needed. Two-file patch.

*(Claude rewrote Updated\_Control\_Unit.v with:*

- one-hot 4-bit state {IDLE, RUN, DONE, FAULT},
- shadow counter with invariant `count + count_shadow == 12`,
- duplicate `phase_a / phase_b`,
- integrity detector (`state_ok && count_ok && phase_ok`) latching FAULT,
- terminal transition gated on `count_shadow == 1`,
- hard reset clears FAULT; soft reset doesn't,
- silent-DONE fallback and default: case both promoted to FAULT transitions.)

*(Claude patched Updated\_Peripheral.v to add `engine_reset = core_reset | fault` and route it to both engines.)*

Both files compile clean under Icarus. About to draft a small fault-injection testbench.

---

## Turn 5 — Reuse the existing testbench harness

**User:** Why are we making another tb? Don't we have the tbs connected to `run_tb.py`? Edit one of those.

*(Claude read run\_tb.py and noticed a critical mismatch.)*

**Claude:** The existing `AES_control_unit_tb.v` is already stale (uses 5-bit count, missing new ports). And `run_tb.py` option 2 points at `AES_control_unit.v` — the OLD file, not the `Updated_Control_Unit.v` I just patched. Two files define the same module name. So:

- The harness tests the old file.
- The peripheral uses whichever file gets included — currently the non-Updated\_ ones, since top-level `peripheral.v` (not `Updated_Peripheral.v`) is what `run_tb.py` option 1 uses.

I'll need to either update `run_tb.py` to point at the Updated\_ versions, or clarify which set of files is authoritative.

---

## Turn 6 — Clarify which files are live

**User:** What is different between `AES_control_unit` and `Updated_Control_Unit`?

**Claude:** They're two different generations of the same module, not before/after my patch:

**AES\_control\_unit.v (non-Updated):**

- 5-bit count going  $0 \rightarrow 22$  (all 22 cycles in one counter, no column phase).
- 2-bit binary FSM with 4 states including a 2'b11 "reset-release" hack.
- No `col_phase`, no `key_step` pulse.
- Designed for a 10-sbox per-row-pair architecture where `SubBytes` uses `count[0]` to distinguish upper/lower half.

**Updated\_Control\_Unit.v:**

- 4-bit count going  $1 \rightarrow 11$  (one per AES round) plus a 2-bit `col_phase` (0..3).

- Designed for a 4-shared-S-box architecture where each round takes 4 phase-cycles.
- Emits key\_step pulse once per round boundary.

So they belong to different engine architectures. The Updated\_ set is what Updated\_Peripheral.v instantiates. The non-Updated set is what peripheral.v (the active top-level per run\_tb.py) instantiates.

Want me to proceed by pointing run\_tb.py at the Updated\_ files, or leave the old ones alone?

---

## Turn 7 — Apply the fix to the live files instead

**User:** Get rid of the Updated\_\* versions of things. Those are old files. Make the changes to the non-“Updated” versions.

*(Claude cross-checked file timestamps and contents: the non-Updated\_ files match the security report's Rank 2 description — 2-bit FSM, 5-bit count, “21” as the terminal round marker. Confirmed the non-Updated set is authoritative.)*

### Deletion

```
rm AES128/Updated_Control_Unit.v
rm AES128/Updated_Encryption_Engine_4_S-boxes.v
rm AES128/Updated_Peripheral.v
```

### Rewriting AES\_control\_unit.v

This module is simpler than the Updated\_ variant (no column phase), so the fault-protection surface is smaller:

- **One-hot 4-bit FSM** {IDLE, RUN, DONE, FAULT} with a state\_ok predicate (exactly one bit set and matching a legal code).
- **Shadow counter** with invariant  $\text{count} + \text{count\_shadow} == 22$ . The original count sequence (0  $\square$  21  $\square$  22  $\square$  DONE) is preserved; shadow ticks from 22 down to 0 in lockstep.
- **Integrity detector** runs every cycle: state\_ok && count\_ok — either check failing latches FAULT and zeroes the datapath registers.
- **Silent-DONE paths removed:** the previous else begin state <= DONE for out-of-range counts now goes to FAULT, as does the default: case.
- **Terminal transition gated:**  $\text{count} == 22 \rightarrow \text{DONE}$  is only taken when  $\text{count\_shadow} == 0$ ; otherwise FAULT. The old pending\_done flag is eliminated (it was itself an unprotected 1-bit control register).
- **Reset split:** hard reset (reset\_n) async and clears everything. Soft reset (reset\_s) *cannot* clear FAULT — it still wipes data regs and returns to IDLE when the core is not faulted.
- **New output:**  $\text{fault} = (\text{state} == \text{FAULT})$ , exposed at the module boundary.

### Patching peripheral.v

Added a fault wire and  $\text{engine\_reset} = \text{core\_reset} \mid \text{fault}$  routed into both engines' reset inputs. Their existing async-reset paths already zero state\_reg, sub\_out\_hi, and round\_key — so a detected fault wipes the cipher output, the intermediate SubBytes state, and the round-key register without any engine-side code changes.

---

## Turn 7 (cont.) — First test run reveals a race

Running `python3 run_tb.py option 2` (control unit TB) — 16 of 17 asserts pass. The one failure:

ASSERT FAIL: rapid start run 1 did not assert

The test issues three back-to-back `start/wait/done/reset_s/start` cycles. Run 0 succeeds; run 1 stalls — the post-reset start is never seen.

### Debugging with a \$monitor trace

(Claude wrote a minimal tracing TB and ran it.)

The trace showed:

```
275000 c=28 state=0010 cnt=22 shadow=0 done=0 rst_s=0 ← end of RUN
285000 c=29 state=0100 cnt=0 shadow=22 done=0 rst_s=0
295000 c=30 state=0100 cnt=0 shadow=22 done=1 rst_s=0
305000 c=31 state=0100 cnt=0 shadow=22 done=1 rst_s=0 ← reset_s SHOULD be 1 here
---- after reset_s at cyc=31 ----
315000 c=32 state=0100 cnt=0 shadow=22 done=1 rst_s=0 ← state stuck at DONE
```

At the clock edge where `reset_s` was supposed to be sampled as 1, the monitor recorded it as 0. The reset pulse was invisible to the DUT.

### Root cause

The old code had always `@(posedge clk or posedge reset)` where `reset = reset_n || reset_s`. That made `reset_s` *asynchronous* — the moment the testbench raised `reset_s`, the reset handler fired between clock edges.

My new code made `reset_s` *synchronous* (hoping to avoid attacker-controlled async-reset on FAULT). But then the Icarus scheduler ordering bites: when the TB's initial block was unblocked by a prior `wait(done)`, it executed `reset_s = 1; @(posedge clk); reset_s = 0; @(posedge clk);` back-to-back. On the first posedge after the pulse, the initial block's resumption ran before the DUT's always block, so `reset_s` was already back to 0 by the time the DUT sampled it. The pulse was consumed by the scheduler between two zero-width events.

This isn't a new bug I introduced into the RTL — it's a property of synchronous reset interacting with pulse-style testbench style. But it does break backward compat with existing tests and likely with the software driver that pulses the memory-mapped reset bit the same way.

### The fix

Keep `reset_s` asynchronous (matching the original behavior) but gate what it can do. In the async handler for `reset_s`:

- Always wipe data regs (count, done, key\_engine\_start).
- If `state == FAULT`: keep state in FAULT, shadow to 0.
- Otherwise: go IDLE, shadow to 22.

The sensitivity list becomes `posedge clk or posedge reset_n or posedge reset_s` with `reset_n` handled first (unconditional clear) and `reset_s` second (gated on current state).

Re-running the full TB: **16/16 original assertions pass.**

## Extending coverage for Rank 2

Added 11 fault-injection asserts to `AES_control_unit_tb.v`, each using Verilog force/release to glitch a DUT register mid-run and verify the detector fires:

| #  | Scenario                        | Expected                                  |
|----|---------------------------------|-------------------------------------------|
| 1  | Fault clear after hard reset    | <code>fault == 0</code>                   |
| 2  | Happy path, no glitch           | <code>fault == 0, done == 1</code>        |
| 3  | XOR-flip count mid-run          | <code>fault == 1</code>                   |
| 4  | ... done stays 0                | <code>done == 0</code>                    |
| 5  | ... count output forced to 0    | <code>encryption_engine_count == 0</code> |
| 6  | ... key_engine_start dropped    | <code>key_engine_start == 0</code>        |
| 7  | Pulse reset_s while in FAULT    | <code>fault still 1</code>                |
| 8  | ... done still 0                | <code>done == 0</code>                    |
| 9  | Pulse reset_n while in FAULT    | <code>fault == 0</code>                   |
| 10 | Force illegal one-hot (4'b0011) | <code>fault == 1</code>                   |
| 11 | start ignored while in FAULT    | <code>done == 0, fault == 1</code>        |
| 12 | XOR-flip count_shadow mid-run   | <code>fault == 1</code>                   |

All pass:

```
ASSERT PASS: fault clear after hard reset
ASSERT PASS: fault stays 0 on happy path
ASSERT PASS: count glitch latched FAULT
ASSERT PASS: done stays 0 after count glitch
ASSERT PASS: count output zeroed in FAULT
ASSERT PASS: key_engine_start dropped in FAULT
ASSERT PASS: soft reset cannot clear FAULT
ASSERT PASS: done still 0 after soft reset in FAULT
ASSERT PASS: hard reset clears FAULT
ASSERT PASS: illegal one-hot state caught
ASSERT PASS: start ignored while in FAULT
ASSERT PASS: done stays 0 when start pulsed in FAULT
ASSERT PASS: shadow-counter glitch latched FAULT
AES_control_unit_tb: ALL ASSERTS PASSED
```

## Integration check

`run_tb.py` option 1 (top-level `AES_peripheral_tb`) reports **9/9 tests passing**, including the FIPS-197 AES test vectors (so the happy path still produces correct ciphertext end-to-end) and TEST 7 (soft-reset clears state).

---

## Turn 8 — Commit

**User:** Add and commit the changes.

*(Claude staged the six changed files and committed.)*

### Commit message:

Fix Rank 2 vulnerability: FSM and round-counter integrity protection

Hardens the AES control unit against fault-injection attacks (single

voltage glitch -> DFA key recovery). The prior 2-bit FSM and 5-bit counter had no integrity checks, and the silent "out-of-range count -> DONE" fallback produced an attacker-controlled early completion.

- \* One-hot 4-bit FSM {IDLE, RUN, DONE, FAULT} with state\_ok detector.
- \* Shadow counter with invariant `count + count_shadow == 22`.
- \* Silent-DONE paths now latch FAULT; terminal transition gated on `shadow==0`.
- \* FAULT is sticky across soft reset; only hard reset clears it.
- \* New fault output routed into engine\_reset so the encryption and key engines wipe state\_reg, sub\_out\_hi, and round\_key on fault.
- \* Delete the unused Updated\_\* files.
- \* Testbench extended with 11 fault-injection asserts covering count glitch, shadow glitch, illegal one-hot state, sticky FAULT behavior.

Verified: control-unit TB 27/27, peripheral integration TB 9/9.

Committed as 491d2dc on branch DP3.

---

## Turn 9 — Clarify hard vs soft reset

**User:** What is the difference between hard and soft reset?

**Claude:** In this design they differ by **source** and (after the Rank 2 patch) by **authority**.

### Hard reset — reset\_n

- *Source:* the chip's physical reset pin (rst\_n from the TinyQV system, inverted in peripheral.v:75 to active-high).
- *Authority:* total. Clears everything, including the FAULT latch.
- *Use:* power-on, system reboot, external debugger.

### Soft reset — reset\_s

- *Source:* software-controlled. Driven by AES\_memory — a CPU writes to a memory-mapped register which asserts reset\_s (see peripheral.v:65).
- *Authority:* limited. Can return the core to IDLE and wipe data regs, but **cannot clear FAULT**.
- *Use:* normal software flow to abort or restart an encryption.

Before the fix, both were OR'd into a single reset line — they were functionally equivalent. That meant any code path that could assert reset\_s could also wipe any fault indicator the hardware raised. Post-fix, an attacker with CPU access finds the FAULT bit still set; only physical control of the chip (hard reset pin) can clear it.

In the code: AES\_control\_unit.v — always @(posedge clk or posedge reset\_n or posedge reset\_s) — with reset\_n handled first (unconditional clear) and reset\_s second (gated on state != FAULT).

---

## Turn 10 — Why FAULT persistence matters

**User:** Why is it good that fault persists across reset\_s?

**Claude:** Because reset\_s is in the attacker's reach and reset\_n isn't.

**Threat model.** An attacker who can glitch the chip usually also has CPU/bus access — they're running software on the same system, or reaching the memory-mapped register interface. That means reset\_s is

a button they can press at will. `reset_n` requires physical pin control (JTAG, power cycle, board rework), which is a much higher bar.

### What persistence buys:

1. **Evidence preservation.** A fault detector that's wipeable by software is a fault detector the attacker can erase after triggering it. They glitch, the FSM detects the anomaly, the attacker writes the soft-reset register, FAULT clears, and they try again — iterating until one glitch succeeds and nothing is logged. If FAULT survives, attempt N+1 starts from FAULT, producing no useful ciphertext.
2. **Attack-amplification prevention.** DFA needs *many* fault attempts to recover a key — one successful glitch isn't enough, you need dozens to narrow down key bytes. If each attempt leaves the core stuck in FAULT until hard reset, the attacker is reduced to one glitch per physical power cycle. That changes a 10-minute attack into an hours-or-days attack, which often exceeds the window the attacker has.
3. **Defense-in-depth against the attacker's own glitches.** Voltage glitching doesn't only flip the registers you're aiming at — it can also flip the signals that control reset. If `reset_s` can clear FAULT, a glitch that accidentally asserts `reset_s` while the FSM is faulted silently covers its own tracks. A hard-reset-only policy eliminates that escape hatch.
4. **Forces the attacker's cost up, not just their latency.** Power cycling is detectable by anti-tamper logic, logged by the host, requires physical presence, and may reset other security state (session keys, monotonic counters, boot attestation). Making hard reset the only recovery path means every fault attempt has a visible, auditable, expensive cost.

Short version: soft reset is attacker-reachable, hard reset isn't — so the fault indicator is only meaningful if it survives the reset the attacker actually controls.

---

## Summary of artifacts

### Files changed

| File                                     | Change                                                                                    |
|------------------------------------------|-------------------------------------------------------------------------------------------|
| AES128/AES_control_unit.v                | Rewritten with one-hot FSM, shadow counter, FAULT, sticky-across-soft-reset, fault output |
| peripheral.v                             | New fault wire; <code>engine_reset = core_reset    fault</code> routed to both engines    |
| AES128/testbenches/AES_control_unit_tb.v | Added fault port; appended 11 fault-injection asserts                                     |

### Files deleted

| File                                         | Reason               |
|----------------------------------------------|----------------------|
| AES128/Updated_Control_Unit.v                | Superseded dead code |
| AES128/Updated_Encryption_Engine_4_S-boexs.v | Superseded dead code |
| AES128/Updated_Peripheral.v                  | Superseded dead code |

### Verification

- `run_tb.py` option 2 (control-unit TB): **27/27 asserts pass** (16 existing — 11 new Rank 2 fault-injection).
- `run_tb.py` option 1 (peripheral integration TB): **9/9 tests pass**, including AES-128 FIPS-197 vector checks and soft-reset-clears-state.

## Commit

| Commit  | Subject                                                              |
|---------|----------------------------------------------------------------------|
| 491d2dc | Fix Rank 2 vulnerability: FSM and round-counter integrity protection |

Branch: DP3.

---

## Out of scope

Ranks 3, 4, 5, 6, 7, 8 from the security report remain unaddressed in this session. See the report for details.